Engagement readiness

What has to be true before the first business-requirements document enters the pipeline for a new engagement, and how the humans run the parts the pipeline does not automate yet. Written after the first full engagement run; the reasoning behind each item is in `docs/design-ledger.md`, and the entry dated 2026-09-02 is the one most of this comes from. If you are reading the markdown: the PDF at `docs/engagement-readiness.pdf` is the version to hand to a team, and `scripts/build-docs-pdf.py` regenerates it.

This is a checklist for the architect setting the engagement up, with the PM and the designer in the room for the parts that are theirs. Running the framework itself — listener, workers, specialist containers — is covered by `docs/local-development.pdf` and each package's README; this document assumes that is done.

---

1. People, and what each one owns

Four roles. One person can hold more than one; every role has to be held by someone who knows it is theirs.

- **PM** — writes the business-requirements document with the `business-requirements-writing` skill; resolves every capability row before the project exists; applies `ready for intake`; confirms the slice map covers the intent; approves each epic's decomposition, including the point estimate on each story; signs off each completed epic by using it.
- **Designer** — works one area at a time, one area ahead of development; produces a design asset for each area and attaches it to that area's epic; confirms the slice map's areas and order at the Intake checkpoint; resolves the design touchpoints in each epic's API map; signs off each completed epic against the running software. Adds cross-cutting rules to the project's design issue as areas reveal them.
- **Architect** — confirms each surface's registry record; writes each surface's `CONVENTIONS.md` with the `conventions-writing` skill; resolves existence in each epic's API map; creates the BRD branch in each surface repo and each epic's branch off it; reviews and merges each epic's PR into the BRD branch after E2E has run and the other two have signed off; merges the BRD branch into `main` at the end.
- **Developers** — read a story and ask the architect questions in its thread before dispatching it; move the story to In Progress, which dispatches the specialist and records the mover as the PR's requested reviewer; review the PR against the acceptance-criteria checklist in its body, ticking what they verified; request changes when needed; merge it into the epic branch.

2. The tracker

The pipeline reads and writes one Linear team. Before the first project:

- **Statuses.** The lanes gate on the tracker's real status names, not the framework's vocabulary. Dispatch fires when a story enters the status named `In Progress`; a story that cannot proceed is

moved back to the status named `Todo`. If the team's workflow uses different names, change the literals in `webhook-listener/src/lanes/specialist-dispatch.ts` and `dispatch-worker/src/activities/move-story-to-todo.ts` — they are engagement configuration and are commented as such. `Backlog` and `Evaluation` must exist as statuses too.

- **Labels.** Create these on the team, exactly as spelled: `ready for intake`, `intake:awaiting-approval`, `ready for eval`; `spec:awaiting-architect`, `spec:awaiting-designer`, `spec:awaiting-answers`, `spec:resolved`; `eval:awaiting-approval`, `eval:awaiting-answers`, `eval:ready`; `design:asset`; one `surface:<name>` per surface (section 3); and the `size:` and `tier:` values from `skills/story-contract/SKILL.md`.
- **The pipeline's bot user**, with an API key (`LINEAR_AGENT_API_KEY`) and its user id in `AGENT_USER_ID`, so the self-comment guard can tell the bot's own comments from a human's.
- **The webhook**, pointed at the deployed listener with the shared secret in `LINEAR_WEBHOOK_SECRET`, sending issue, comment, and project events.
- **Linear's GitHub integration** connected to the surface repos, so a merged PR moves its story to Done. The dependency check reads tracker status; without this, every dependent story stays blocked until someone moves the merged one by hand.

3. The surfaces

A surface is a place work happens: a repo, or a project inside one. For each surface the engagement will touch:

- **A `surface:<name>` label** on the team. Names are the engagement's own. Keep them short and stable; renaming a surface mid-engagement cost the first engagement roughly fifty written stories.
- **A record in the surface registry.** The registry is a project document titled `Surfaces`, one per engagement, holding a record per surface: repo, ref, the directory within the repo, the conventions file, any mandatory skills, and status. An epic can carry a `Surfaces (override)` document for differences that hold only for that epic. The agents write both — the Specification Agent proposes a record from what it can read when a surface has none, the architect confirms in the thread, the agent writes it. The architect never types the format; only `github` is a supported host today. Know where every surface lives before the first epic so each proposal is confirmed in one reply.
- **Real code at that ref**, readable enough that Specification can state the runtime and framework as a fact and see at least one representative pattern. A greenfield surface needs a starter solution before the pipeline can map it; the Specification Agent blocks until it exists.
- **A `CONVENTIONS.md` at the surface root on that ref.** Mandatory since the specialist definition went generic about how to build: it is the only place house style, test levels, and how each suite is invoked are written down. Write it with the `conventions-writing` skill. Decompose refuses to cut stories for a surface without one.
- **CI that runs the surface's unit tests on every pull request.** The specialist sandbox carries Node, Python (with Poetry) and .NET, and the specialist runs its own tests there, but CI is the independent check on that. A stack outside those three cannot be built or tested in the sandbox at all, and CI is then the only check. Make sure it is there.

- **The GitHub token** (`GITHUB_TOKEN` for `dispatch-worker` and the sandbox) can read the repo, create branches, and open pull requests. It also needs push access to the repository, because `dispatch-worker` reads other users' collaborator permission to decide whose "request changes" review may start a revision round.
- **A reviewer mapping.** `REVIEWER_EMAIL_TO_GITHUB_LOGIN` in `dispatch-worker`'s environment maps each developer's Linear email to their GitHub login, so the person who moves a story becomes the PR's requested reviewer. An unmapped developer still gets a PR, but nobody is asked to review it; the PR says so when it opens. Revision rounds do not depend on the mapping: a "request changes" review from anyone with write access to the repository starts one.

4. The requirements session

Run `business-requirements-writing` with the PM, and with the designer in the room when possible. What comes out:

- A BRD with every capability row resolved `in-scope` or `out` — no `confirm` row reaches the tracker.
- **If the designer is present**, an optional "Design order of work" section: the areas the designer intends to work, in order, each naming the capabilities it covers. Intake slices to match these areas and uses the order as its starting point. If the designer is absent, leave the section out; Intake proposes an order from the dependency graph and asks the designer to confirm it.
- **A thin design issue**, labeled `design:asset`, holding only cross-cutting experience rules the evidence already states. It is not seeded with per-area design specifics. A design issue with a few rules or none is the expected state at this point.
- Evidence placed where a downstream agent can open it: text as project documents, binaries on the evidence issue, and a legend in the project description saying what each artifact is and which is authoritative.
- The PM applies `ready for intake`. Nobody else does.

5. The per-area rhythm

This is the loop the engagement runs in. The designer is always one area ahead of the developers.

1. **Intake slices the whole BRD** into epics whose boundaries are the designer's areas, and posts the slice map with a proposed order and every place the designer's order and the dependency order disagree. The PM confirms coverage; the designer confirms the areas and the order. The approval names which epics release now — normally the one or two the designer is starting with. The rest wait in Backlog.
2. **The designer designs the first area** and attaches the asset (Figma frames, prototype additions, a findings document, a review transcript) to that epic's Evidence section. Text becomes a project document; binaries attach; links are links.
3. **The architect creates the epic's branch** off the BRD branch in each surface repo the epic touches. Story branches are created by the app; epic branches are not, and dispatch fails visibly without one.

4 **The epic is released to Evaluation.** Specification blocks until it finds design evidence on the epic (or the designer gate is waived for an epic with no user-facing behavior), then drafts the API map: design touchpoints for the designer, technical touchpoints for the architect, references in a footer. Each reviewer resolves their rows in the thread, one row at a time. Decompose cuts stories once both gates clear, and proposes a point estimate for each; the PM approves the decomposition and corrects any estimate; the stories land in To-Do. Each story's `Blocking dependencies` section is what dispatch checks. Decompose mirrors it as the tracker's blocked-by relations so the order is visible on the board, but the section is the source of truth. Decompose also writes a `Definition of done — coverage` checklist into the epic's description, mapping each line of the definition of done to the stories behind it.

5 **Developers pick stories up** by reading the story, asking the architect in the thread, and moving the story to In Progress. The specialist runs and opens a PR against the epic branch whose body lists each acceptance criterion as an unticked checkbox, with the code and test behind it. The mover reviews the diff against that list, ticks what they verified, and merges. If they submit a "request changes" review, the specialist is sent back to address it on the same PR, up to three rounds; the app posts a note on the PR as each round starts and finishes, and when the rounds run out. The story stays In Progress while the PR is open.

6 **The designer is already on the next area** while this happens. Its epic is released when its design lands — not before.

6. Epic completion — a human procedure, for now

The pipeline does not execute E2E suites or record epic sign-off; that automation is deliberately parked, and humans run it. When every story under an epic has merged into the epic branch:

1 **The architect opens the epic's PR** into the BRD branch.

2 **The architect stands the BRD branch up** with that epic's work included — the same environment the epic's E2E story was written against — and **runs the E2E suite**: this epic's tests and every previously merged epic's. A failure routes to whoever reviews the epic PR; it does not go back through a specialist.

3 **Three people sign off**, in any order: the architect (everything merged, E2E green, no open blockers), the designer (the area's design intent holds in the running software), and the PM (it works, and the PM can speak to it). Each ticks the lines of the epic's `Definition of done — coverage` checklist they verified and replies in the epic's thread. An unticked acceptance criterion from a story PR is already in the epic branch's history, since story PR bodies become the squash-merge commit messages; read those when opening the epic's PR.

4 **The architect merges the epic PR** and redeploys. The environment is cumulative from here; the next epic's E2E runs against it.

Do not move on from an epic before step 3. The first engagement ran E2E only after every epic was done and found gaps everywhere; the whole point of this procedure is that each area is tested and signed off before the next one is built on it.

7. Closing the BRD

When the last epic has merged, the BRD branch gets the same three-way sign-off — architect, designer, PM — and the architect merges it into `main` and moves the project to Done by hand. Nothing automates either step.

There is no closing epic. The design ledger describes one, holding cross-epic E2E stories, but it is parked and nothing creates it. With E2E run cumulatively at every epic completion, cross-epic flows are already exercised along the way. If the engagement wants one anyway, the architect creates it by hand as the last epic in the project, depending on every other epic.

8. What to expect, honestly

- **The sandbox runs Node, Python and .NET.** On those stacks the specialist runs its own tests and the existing ones before opening the PR; CI is still the independent check. It cannot run the team's full local environment, which is why E2E runs by hand at epic completion.
- **Rebuilds of a story are not idempotent.** Two runs of the same story can make opposite decisions, so review before closing a PR unmerged. Closing one moves the story back to Todo. Before it can be dispatched again, delete its story branch: the app refuses to dispatch onto a story branch that still carries commits from the abandoned attempt, and says so on the story.
- **A story that cannot proceed goes back to Todo with a comment.** Read the comment; the pipeline does not retry on its own.
- **Revision rounds are capped at three, each deliberately short.** Feedback too big for a round is a story change, and the specialist will say so and recommend closing the PR. When the rounds run out, the PR stays open for the reviewer to finish by hand.
- **Formats the pipeline parses are strict.** The surface registry's `surfaces` block, the `## References` footer, the blocking-dependency bullets. The agents write them; a human should not need to, and a malformed registry record is reported as a bug rather than worked around.
- **Nothing here references this engagement by name.** `CONTRIBUTING.md` explains why; the check runs in CI. Engagement-specific configuration belongs in environment variables and untracked files, not in this repository.